

Scientific Computing

11/12/25

* This is the discipline that uses mathematical models, numerical methods & high performance computing.

* This enables to simulate, model & optimize.

Applications

- AI
- Mathematics
- Bio, DNA
- Genetics
- Weather
- Neural Networks.

Hardware Acceleration

* Enhancement of energy efficiency by delegating computational tasks from general purpose processors to dedicated hardware components.

* GPU's, TPU's, FPGA's are hardware accelerators

Uses

- Improve performance
- Reduce execution time
- Process large data faster.
- Enable real-time AI, graphics, & scientific computing.
- Increase efficiency & reduce energy usage.

* GPU (graphics processing units) accelerators significantly reduce the amount of time a computer needs to run multiple programs.

* CPU executing multi threads at the same time accessing the same cache, will not deliver high speed.

* Where as in multi-core system, multiple CPU cores each of which processing a task at a different cache, executes at higher speed.

16/12/25

Processing - Performing the frequently executed operations in the fastest way possible in this context.

* Special hardwares used today for different purposes are,

→ Graphics Processing Units (GPU)

* Excel at parallel processing tasks

* Used for graphics rendering & scientific simulation

→ Digital Signal Processors (DSP's)

* Optimized for processing the digital signals in real time.

* Ideal for audio, video & telecommunication applications.

Vector Processing Units (VPUs)

- * Used when performing operations on large arrays of data.
 - * Hardware accelerators are used to perform custom ^{process} tasks.
- ## Field Programmable Gate Arrays (FPGAs)
- * Offers reconfigurable hardware that can be customized to accelerate specific algorithms or tasks.

Application Specific Integrated Circuits (ASICs)

- * These are custom designed.
- * Offers high efficiency ~~and~~ in performing a specialized task, with optimization.

Network Interface Cards (NICs)

- * Performs network-related tasks such as packet processing, encryption, compression & reducing CPU overhead & latency.
- * Done through offload engines.

Caching Mechanisms

- * Cache is shorter storage of data & CPUs have numerous cache resources to store data during processing.

⇒ CPU caches

- * L1, L2, L3

- * stores frequently accessed data which is closer to the processor.
- * reduces memory access latency.
- * increases the processing speed.

→ Graphics memory (VRAM).

- * This is in GPU's
- * serves as a high speed cache

* CPU executing a group of ~~extro~~ instructions simultaneously is called parallel processing.

* CPU specific assembly language consists of instructions, these instructions are converted to binary code.

* CPUs utilize → instruction pipelining & superscalar execution for the parallel instruction execution.

18/12/25

Hardware Prefetching

- * CPUs employ prefetching mechanisms to anticipate memory access patterns & fetch data into caches before it is needed.
- * Improves processing speed.
- * This means performing certain set of actions in advance, or before the time at which these set of actions is executed.

→ Video & Image Process

- * Video encoding/decoding.

- * 4K/8K video editing.

- * VPUs & GPUs are used.

→ AUTONOMOUS SYSTEMS.

- * Self-driving Cars.

- * Robotics.

- * Drone Navigation.

→ Accelerating Medical & Healthcare applications.

- * MRI/CT image reconstruction.

- * Protein structure prediction.

- * Drug discovery simulations.

- * GPUs, TPUs, FPGAs are used.

High performance computing

- * Use of supercomputers, clusters & high speed computing resources to solve complex scientific, engineering & data intensive problems.

- * HPC systems can perform trillions or even quadrillions of operations per second.

→ High Parallelism.

→ High memory bandwidth

→ Increase performance proportionally.

→ Speed in computing ~~comp~~ matrix & vector computations.

→ These are used in scientific, engineering, Artificial, Bigdata Analytics, Medical & biological Applications.

Role of Scientific Computing in Engineering

Engineering Applications | Roles

- 1) Mechanical $\Rightarrow$ Stress analysis, thermal simulations, CFD.
- 2) Civil $\Rightarrow$ Structural analysis, earthquake modeling.
- 3) Electrical $\Rightarrow$ Signal Processing.
- 4) Chemical $\Rightarrow$ Reaction kinetics.
- 5) Aerospace $\Rightarrow$ Aerodynamics.
- 6) Biomedical $\Rightarrow$ Medical imaging.

Roles

1) Simulation & Modeling

* Scientific computing is to create & execute simulations based on mathematical models.

Ex:- Climate Science $\Rightarrow$ Researchers use High Performance Computing to run complex earth system models.

2) Data Analysis & Prediction (Biomedical Research)

* Scientific computing handles enormous datasets generated in modern biological & medical research.

Ex:- Genomics & Proteomics
Molecular Dynamics
Predictive Analytics

3) Uncertainty Quantification & Optimization

* Scientific computing provides crucial methods for validating models & optimizing designs.

Ex: Optimizing the thrust efficiency of a rocket engine.

Engineering Applications

1) Virtual Prototyping & Simulation

→ Computational Fluid Dynamics (CFD):

* we use CFD to model & analyze the flow of fluids.

→ Aerospace

→ Automotive

→ Civil Engineering

→ Finite Element Analysis

→ Acoustic & Electromagnetics.

2) Optimization & Design Exploration

→ Design & Topology Optimization.

3) Safety & Risk Assessment

→ Nuclear & Power Engineering.

→ Disaster Modeling.

→ Crash Safety.

4) Integration with AI

- * ~~S&C~~ S.C. models are increasingly merge with data from sensors & Machine Learning to create advanced systems.

AI in S.C

1) Deep Learning

- Floating-Point Arithmetic operations.
- High-Performance Computing.
- Numerical Optimization.

2) Scientific Machine Learning

This represents the deep fusion of Scientific computing & machine learning

- Physics-Informed Neural Networks.
- Model Acceleration.
- Automated Model Discovery.
- Solving Inverse Problems.

3) Data Management & Interpretation

S.C is crucial for handling the immense, complex, & often multi-modal data generated & consumed by AI systems.

Accelerator Programming

- * This involves writing software that uses specialized hardware such as GPUs or FPGAs to speed up computation.

Features

- High computational Throughput.
- Energy efficiency.
- Massive parallel execution.
- Reduced execution time.
- Cost efficiency at scale.
- Parallelism

Types of Parallelism

- 1) Data Parallelism ⇒ Same operation on different data.
- 2) Task Parallelism ⇒ Different tasks run concurrently.
- 3) Pipeline Parallelism ⇒ Stages of computation overlap.
- 4) Instruction-Level Parallelism (ILP) ⇒ Multiple instructions per cycle.

Accelerator Programming Models & Tools

- CUDA Compute Unified Device Architecture
- * Platform & API for general-purpose computing on its GPUs. This provides low-level control over the hardware.

Ex: NVIDIA GPU Programming.

→ Open CL (Open Computing Language)

* An open standard framework that supports cross-platform, parallel computing on various hardware accelerators (GPU's, FPGA's, etc.).

* This is platform independent accelerator programming.

→ Directive-based models (OpenACC, OpenMP)

* These models use or offer a higher level of abstraction and ease of use, even though performance can be highly dependent on compiler capabilities.

→ Domain-specific Languages (DSLs) & Frameworks

* This abstracts away the complexity of programming the underlying hardware for specific applications.

→ Data-Parallel Libraries (Accelerator API).

* Uses functional programming models to perform array-processing operations.

→ Task-Based Models (Compss, Celerity)

* Breaks applications into tasks with dependency tracking. Allows for asynchronous execution.

→ Unified / Heterogeneous Models

- * Approaches that treat accelerators as I/O devices or specialized nodes within a single address space.
- * Unifies programming across different hardware types.

→ Unified / Heterogeneous Models

SYCL:- A single source C++ standard for heterogeneous computing, allowing host & device code to be in the same file.

- * For high throughput & massive parallelism we use GPU models (CUDA, OpenCL).
- * Low Latency & energy efficiency, FPGA models are used.
- * Portability across heterogeneous systems is done by through SYCL.
- * Framework based accelerator models are used for rapid development for AI workloads.

Data Transfer Overhead

- Host - to - Device (H2D) ⇒ Copying data from CPU to accelerator.
- Device - to - Host (D2H) ⇒ Data transfer from the GPU back to the CPU.

- * ^{In} Unified memory management system where the CPU & GPU share a single, logical memory space.

↑ Pinned memory is host memory that is locked, allowing faster data transfer b/w CPU & GPU.

Parameter	H2D	D2H
Direction	Host → Device	Device → Host
Purpose	Send input Data	Retrieves results
When	Before Computation	After computation
Common Bottleneck	PCIe bandwidth	PCIe latency
Optimization	Pinned memory, async copy	Async copy, batching

* Latency ⇒ Time delay between initiating an operation and the moment the first result becomes available.

Hardware Acceleration

Types

- 1) GPU's (Graphics Processing Units)
- 2) FPGA's (Field Programmable Gate Array's)
- 3) ASIC's (Application-specific Integrated circuit)
- 4) NPU's (Neural Processing Units)
- 5) TPU's (Tensor Processing Units - Google)
- 6) VPU's (Vision Processing units)

TPU (Tensor)

- * Designed by Google for massive cloud-based AI training. This passes data through a grid of processors like blood through a heart to minimize memory access.

NPU (Neural)

- * Found in smartphones & IoT devices. They are optimized for energy efficiency & "on-device" AI.

VPU

- * A subset of NPUs specifically optimized for computer vision tasks.

- Deep Learning Training ⇒ GPU/TPU
- Real-time AI Inference ⇒ NPU/TPU
- High-Throughput Cloud AI ⇒ TPU/GPU clusters
- AI for Embedded/IoT systems ⇒ FPGA/NPU/VPU
- Specialized AI deployment at scale ⇒ ASIC
- Reconfigurable AI Accelerators ⇒ FPGA
- CUDA (Compute unified device architecture Platform)

- * This is a parallel computing platform.

- * This is for GPU computing.

- * CUDA allowed thousands of GPU cores to execute tasks in parallel.

- * CUDA transformed GPUs into primary hardware accelerators.
- * Intensive, loops to be executed on thousands of GPU threads simultaneously.

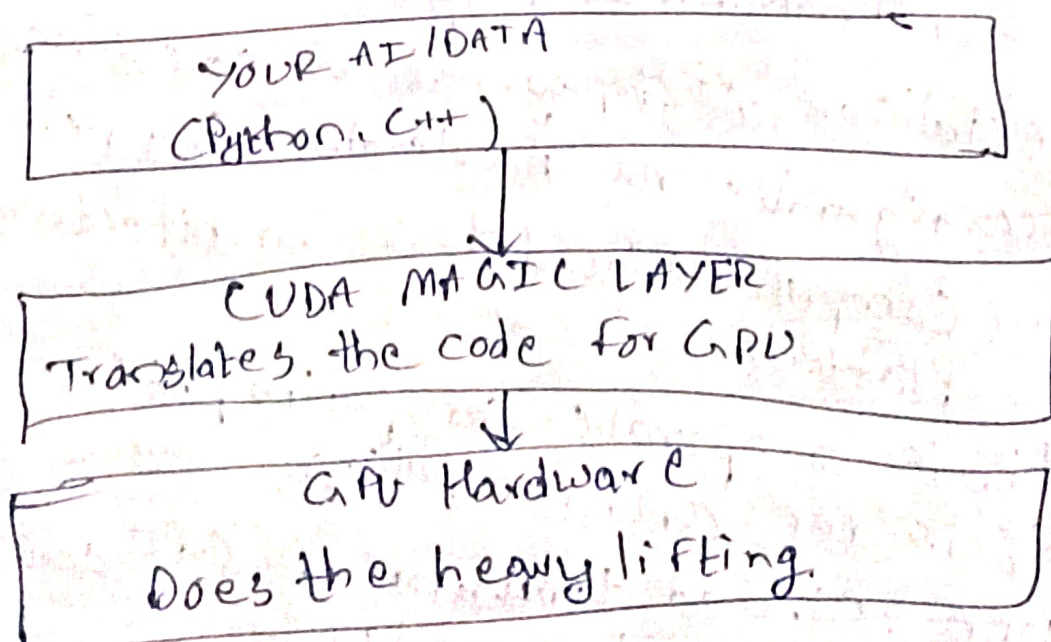
Characteristics

- Massive Parallelism.
- Memory Hierarchy Optimization.
- AI Acceleration Features.
- Data Analytics Acceleration.

Massive Parallelism

- * Thousands of lightweight threads organized into warps, blocks, & grids.
- Threads $\Rightarrow$ Smallest execution unit.
- Warps $\Rightarrow$ Groups of threads.
- Blocks $\Rightarrow$ Collection of warps.
- Grid $\Rightarrow$ Collection of blocks.

Working



* Workloads like Netflix, Youtube & self-driving cars,

Compute-Intensive Workloads

- * These workloads require large amounts of CPU/GPU processing power with relatively less data movement.
- * These ~~calculations~~ workloads spend most of their time performing calculations rather than reading or writing data.

Key characteristics

- Heavy mathematical or logical operations.
- High CPU/GPU utilization.
- Minimal disk (or) network I/O.
- * Performance depends on processor speed, cores & parallelism.

Ex- Scientific Simulations, Cryptographic Operations, 3D rendering & Ray Tracing, Financial Modeling, Numerical computation

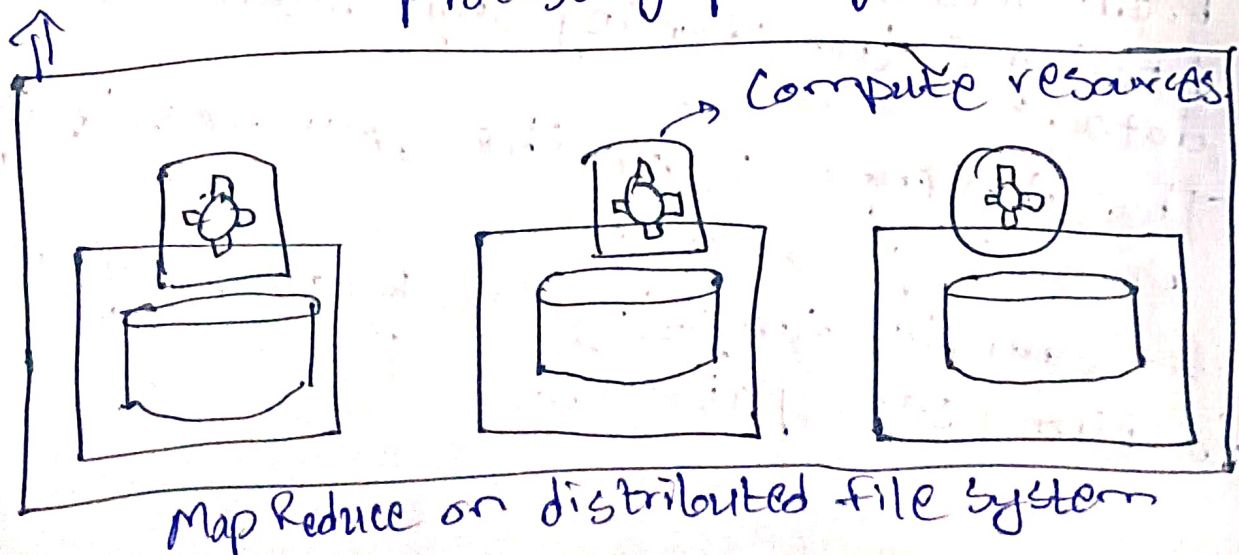
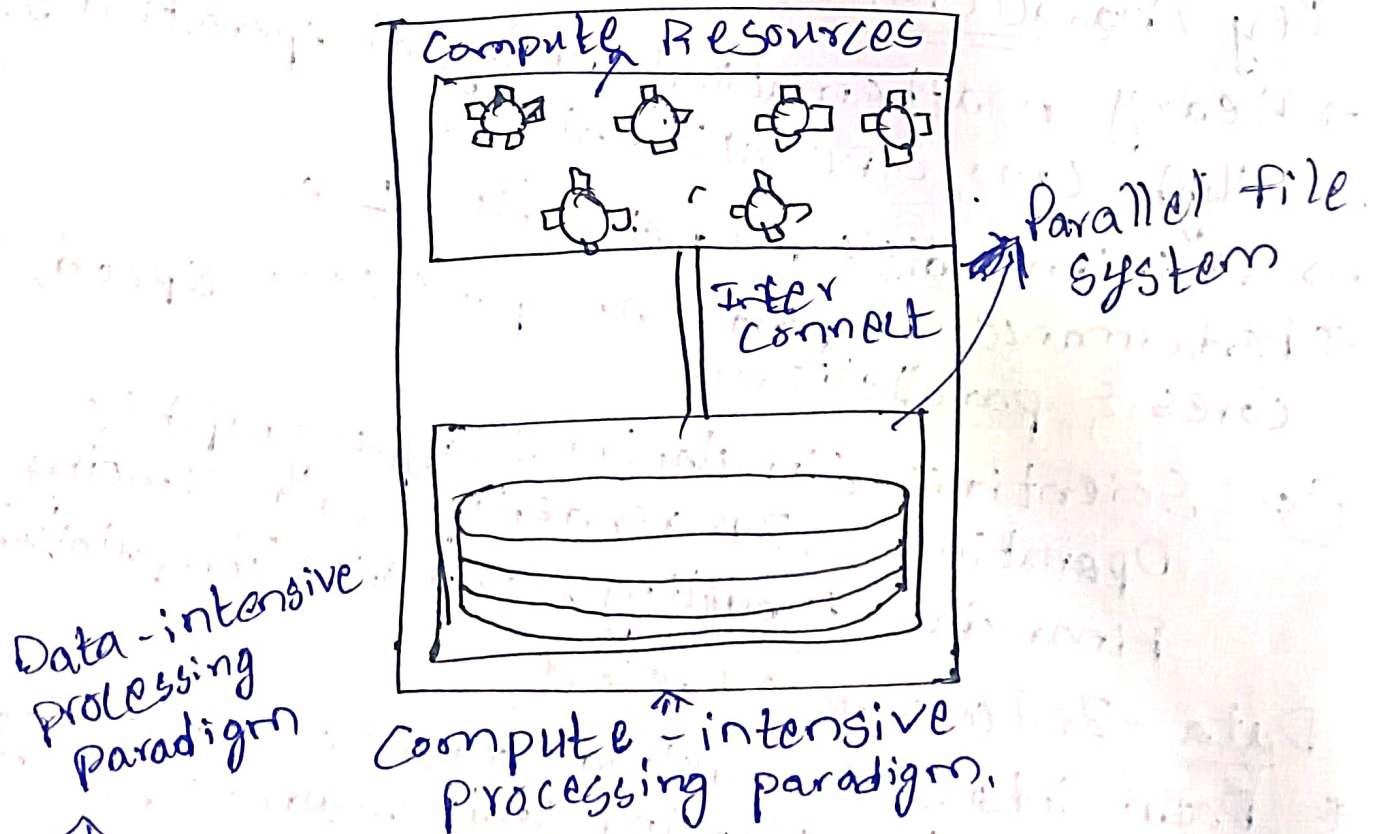
Data-Intensive Workloads

- * Data-intensive workloads tasks on processing, storing & transferring large volumes of data.
- * The challenge is on the volume, velocity, or variety of data.
- * For example, Analyzing tera bytes of log files to find usage patterns.

Key Characteristics

- * Large datasets
- * Frequent disk & network access.
- * Computation per data unit is relatively low
- * Performance depends on storage, memory, bandwidth.

Ex:- Big data Analytics, Social media analytics
Video streaming Platforms, Large scale ETL
Pipelines, Data lakes & warehouses.



* Other workload types are memory-intensive, I/O-intensive, Hybrid workloads.

Arithmetic Intensity

$$\text{Arithmetic Intensity} = \frac{\text{No of arithmetic Operations}}{\text{Amount of data moved.}}$$

* Expressed as FLOP's per byte.
(floating-point op's per bytes of memory transferred).

Ex:- Performs 1,000,000 operations.
Transfers 500,000 bytes of data.

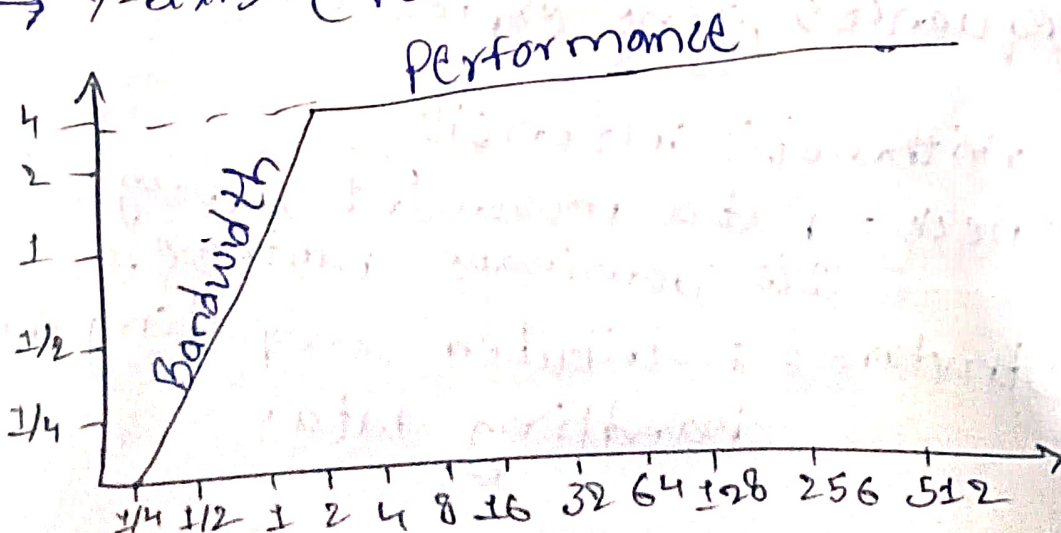
$$A.I = \frac{1,000,000}{500,000} = 2 \text{ ops/byte.}$$

Roofline model

* This is a visual performance model used to analyze & predict the maximum achievable performance of a program on a hardware platform.

→ X-axis (Arithmetic Intensity).

→ Y-axis (Performance).



- Memory Bandwidth Roof (Slanted Line)
- * Represents the max performance limited by memory bandwidth.

$$\text{Performance} = \text{Arithmetic Intensity} \times \text{Memory Bandwidth}$$

↓
for memory-bound workloads.

- Compute Roof (Horizontal Line)
- * Represents the peak computational capability of the processor.
- AI - Training Computing Intensive
- * AI model training, especially deep learning, involves performing millions or billions of mathematical operations.
- High Arithmetic Intensity.
- Bottleneck = CPU/GPU compute power.
- Infrastructure focus ⇒ GPUs/TPUs are used

Genomics data - intensive

- * These involve processing huge datasets. (DNA sequences, gene expression data).
- Low Arithmetic intensity.
- Bottleneck = Data movement, storage, & I/O dominate runtime.
- Infrastructure = Distributed computing for handling data.

operations

- Large - Scale Sequencing data.
- Data Analysis (Complex).
- High-Throughput Experiments.

* High + performance storage systems, distributed computing, optimized algorithms, are used for accelerating genomics analysis.

→ Climate modeling is both compute-intensive & data-intensive. For this HPC (High - Performance Computing) is required.

* Climate modeling is data-intensive because of large input/output datasets & is compute-intensive due to heavy simulations.